



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS**

FUNCIONES SINCRONAS Y ASINCRONAS EN JAVASCRIPT

Presentado a: Yoly Alexandra Guevara Muñoz
Por Aprendiz: Miguel Angel Rivera
Ficha: 3312932

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día 15 de mes 09 del año 2026



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

1. **Objetivo Comprender la diferencia entre las funciones síncronas y asíncronas en JavaScript, identificando su funcionamiento, características, ventajas, casos de uso y situaciones en las que es conveniente utilizar cada una.**
2. **solucion investigación**

¿Qué es una función síncrona?

Es una función cuya ejecución ocurre de manera inmediata y en el mismo orden en que fue escrita en el código. El programa espera a que la función termine por completo antes de pasar a la siguiente instrucción.

¿Qué es una función asíncrona?

Es una función que inicia una tarea y permite que el programa continúe ejecutando otras instrucciones sin esperar a que dicha tarea termine.

¿Cómo se ejecuta una función síncrona?

Se ejecuta línea por línea, de forma secuencial, dentro del mismo hilo principal de JavaScript. Cada instrucción debe finalizar antes de que la siguiente comience.

¿Cómo se ejecuta una función asíncrona?

Se ejecuta línea por línea, de forma secuencial, dentro del mismo hilo principal de JavaScript. Cada instrucción debe finalizar antes de que la siguiente comience.

¿Cómo se ejecuta una función asíncrona?

Se ejecuta iniciando la tarea y delegando su finalización al 'Event Loop'. Cuando la tarea termina, se coloca en la cola de tareas y se ejecuta cuando el hilo principal queda libre.

¿Qué significa que una operación sea bloqueante?

que detiene por completo la ejecución del programa hasta que termina, impidiendo que cualquier otra instrucción se procese mientras tanto.

¿Qué significa que una operación sea bloqueante?

Significa que detiene por completo la ejecución del programa hasta que termina, impidiendo que cualquier otra instrucción se procese mientras tanto.

¿Qué significa que una operación sea no bloqueante?



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

Significa que el programa puede seguir ejecutando otras tareas mientras esa operación se completa en segundo plano, sin detener el flujo principal.

¿Qué es una Promise?

Es un objeto de JavaScript que representa el resultado de una operación asíncrona. Puede estar en tres estados: pendiente, cumplida o rechazada.

¿Para qué sirven async y await?

async convierte una función en asíncrona, haciendo que siempre retorne una Promise. await se usa dentro de una función async para pausar su ejecución hasta que la Promise se resuelva.

¿Qué problemas pueden solucionarse utilizando programación asíncrona?

evita que la aplicación se congele mientras espera operaciones lentas, como peticiones a servidores, lectura de archivos o consultas a bases de datos, mejorando la experiencia del usuario y el aprovechamiento de los recursos del sistema.

3.

Característica	Funciones Síncronas	Funciones Asíncronas
¿Cómo se ejecutan?	Una instrucción después de otra, en el mismo hilo de ejecución.	Se inician y continúan ejecutándose en segundo plano mientras el programa sigue con otras tareas.
Orden de ejecución	Secuencial: en el orden en que aparecen en el código.	No secuencial: el resultado se recibe cuando la operación termina, sin importar el orden en que se escribió.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

¿Bloquean la ejecución?	Sí, bloquean el hilo hasta que la instrucción termina.	No bloquean, el programa continúa ejecutando otras líneas mientras espera el resultado.
¿Esperan una operación?	Esperan de forma inmediata y bloqueante.	Esperan de forma diferida, mediante Promises o async/await.
Tiempo de respuesta	Inmediato, depende solo de la velocidad del procesador.	Variable, depende de un evento externo.
Uso de Promise	No.	Sí, es la base para manejar resultados futuros de operaciones asíncronas.
Uso de async/await	No.	Sí, permite escribir código asíncrono con apariencia secuencial.
Ejemplos	Suma de números, validaciones simples, operaciones matemáticas.	Peticiones a una API, lectura de archivos, consultas a bases de datos.
¿Cuándo se utilizan?	Cuando la tarea es rápida y no depende de recursos externos.	Cuando la tarea puede tardar tiempo indeterminado.
Ventajas	Código simple y fácil de seguir.	No bloquea la interfaz ni el resto del programa; mejora la experiencia del usuario.



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS**

Desventajas	Puede congelar la aplicación si la tarea es lenta.	Mas complejidad en el manejo de errores y el orden de ejecución.
--------------------	--	--

4. solución

1. el mensaje que aparece primero es el inicio
2. el segundo mensaje que aparece es: “hola aprendiz”
3. el tercer mensaje es: “Fin”
4. si, porque es una función síncrona.
5. Este código es síncrono.
6. porque todas las instrucciones se ejecutan una detrás de otra, en el mismo orden en como están escritas.

5. solución

1. el primer mensaje es: “inicio”
2. el segundo mensaje es: “Fin”
3. el tercer mensaje es: “proceso terminado”
4. Porque `setTimeout` es una función asíncrona.
5. No, El programa sigue ejecutando cualquier otra instrucción disponible.
6. el no bloqueo, mientras se ejecuta el `setTimeout` la ejecución sigue.

6. solución

El orden es: A,C,E,B,D.

la B no aparece después del A porque, aunque el `setTimeout` sea 0, el `setTimeout` manda su callback al event loop, sin importar el retraso indicado.

7. solución

1. devuelve una promise que se resuelve como “usuario encontrado” después de 2000 ms.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

2. es la función que marca la promise como cumplida y entrega el valor final.
3. `mostrarUsuario()`
4. pausa la ejecución de la función `async` hasta que la `promise` se resuelva.
5. La función `mostrarUsuario()` queda en pausa, pero el resto del programa sigue funcionando con normalidad.
6. Porque permiten escribir el código asíncrono con una estructura secuencial, parecido al código síncrono tradicional.
7. resultado tomaría el valor de la `Promise` sin resolver (un objeto `Promise` pendiente) en lugar del texto "Usuario encontrado", y los `console.log` siguientes se ejecutarían de inmediato sin esperar el resultado real.

8. solución

Situación	S / A	¿Por qué?
Sumar dos números	S	se resuelve en el mismo instante.
Validar que un campo no esté vacío	S	es una comprobación instantánea de datos que ya están en memoria.
Consultar información de una API	A	bloquear la aplicación mientras se espera afectará la aplicación.
Consultar una base de datos	A	el tiempo de respuesta depende del motor de bases de datos, por lo que no se debe bloquear el programa.
Esperar una respuesta de un servidor	A	el servidor puede tardar un tiempo



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS**

Situación	S / A	¿Por qué?
		indeterminado en responder.
Calcular el total de una factura	S	es una operación simple.
Leer información de un archivo de manera asíncrona	A	La operación es asíncrona.
Realizar una animación con retraso	A	el retraso se maneja con temporizadores que se ejecutan en segundo plano.
Procesar una operación que depende de una API	A	Debe esperar el resultado de un recurso externo antes de continuar.
Mostrar el resultado de una operación matemática	S	El resultado ya está calculado y se muestra de inmediato.

9. solución

- usaría una función asíncrona
- porque la respuesta del servidor puede tardar un tiempo indeterminado.
- la interfaz dejaría de responder, el usuario no podría hacer clic en nada, navegar por la pagina ni cancelar la accion.
- promise:** representa el resultado futuro de la solicitud del servidor.
 - fetch:** es la función que realiza la solicitud HTTP al servidor y devuelve un promise con la respuesta.
 - async:** se usa para declarar la funcion que maneja la consulta asíncrona.
 - await:** pausa la funcion async hasta que la promise de fetch se resuelva, para trabajar con los datos ya recibidos.

10. solución



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

```
JS index2.js > ...
1  function consultarUsuario() {
2      return new Promise((resolve) => {
3          setTimeout(() => {
4              resolve("Usuario encontrado");
5          }, 3000);
6      });
7  }
8
9  async function main() {
10     console.log("Consultando usuario...");
11     const resultado = await consultarUsuario();
12     console.log(resultado);
13     console.log("Proceso terminado");
14 }
15
16 main();
17
```

Problems Output Debug Console Terminal Ports

PS C:\Users\PC\Desktop\html-css\DOM\sweetAlert2> node index2.js
Consultando usuario...
Usuario encontrado
Proceso terminado
PS C:\Users\PC\Desktop\html-css\DOM\sweetAlert2>

11. solución

1. La diferencia es que la función síncrona se ejecuta en orden, bloqueando el programa hasta terminar. Mientras que la asíncrona puede iniciar una tarea y deja que el programa siga ejecutando otras tareas mientras espera el resultado que llegará.
2. porque el tiempo que se demora en responder el servidor es indeterminable.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

3. `async` y `await` hacen que el código asíncrono se lea de forma normal o síncrona, esto evita que se aniden muchos `then`.
4. no necesariamente, en javascript las funciones asíncronas se siguen ejecutando en el mismo hilo principal, y se apoyan en el event loop para que cuando ya este vacío de tareas ahora si ejecute la promise.
5.
 - consultar datos de un usuario con una API.
 - consultar pedidos de un cliente desde el servidor sin bloquear la página.
 - pedir datos a una API externa con `fetch`.